

Lecture 1: Arrays, Linked Lists, Stacks & Queues

CS301 — Algorithms & Data Structures

Department of Computer Science

1. Introduction to Data Structures

A data structure is a particular way of organizing data in a computer so that it can be used effectively. The idea is to reduce the space and time complexities of different tasks. Data structures are the backbone of software development and algorithmic thinking.

This course covers fundamental data structures: arrays, linked lists, stacks, queues, trees, graphs, and hash tables. Understanding these is essential for writing efficient programs.

2. Arrays

An array is a collection of items stored at contiguous memory locations. The idea is to store multiple items of the same type together. This makes it easier to calculate the position of each element by simply adding an offset to a base value (the memory location of the first element of the array).

Array Operations and Complexities: - Access: $O(1)$ — direct indexing - Search: $O(n)$ — linear scan for unsorted, $O(\log n)$ for sorted (binary search) - Insert: $O(n)$ — may need to shift elements - Delete: $O(n)$ — may need to shift elements

Dynamic Arrays (e.g., Python lists, Java ArrayList) automatically resize when capacity is exceeded, typically doubling in size, giving amortized $O(1)$ append. The resizing operation itself is $O(n)$, but spread over n operations gives $O(1)$ amortized cost.

3. Linked Lists

A linked list is a linear data structure where elements are stored in nodes. Each node contains: - data: the stored value - next: a pointer/reference to the next node

Types of Linked Lists: 1. Singly Linked List: each node points to the next node only 2. Doubly Linked List: each node points to both next and previous nodes 3. Circular Linked List: last node points back to the first node

Complexities: - Access: $O(n)$ — must traverse from head - Search: $O(n)$ - Insert (at head): $O(1)$ - Insert (at tail, with tail pointer): $O(1)$ - Delete (with predecessor known): $O(1)$

Linked lists excel when frequent insertions/deletions occur and random access is not needed. Arrays excel when random access is frequent. This tradeoff is fundamental in algorithm design.

4. Stacks

A stack is a linear data structure that follows the LIFO (Last In, First Out) principle. Think of a stack of plates: you add a plate to the top and remove from the top.

Core Operations: - push(x): add element x to top — $O(1)$ - pop(): remove top element — $O(1)$ - peek(): view top without removing — $O(1)$ - isEmpty(): check if empty — $O(1)$

Implementation: stacks can be implemented using arrays (with a top pointer) or linked lists (push/pop at head).

Applications of Stacks: 1. Function call stack (activation records) 2. Expression evaluation (infix $\rightarrow$ postfix, postfix evaluation) 3. Backtracking algorithms (DFS, maze solving) 4. Undo mechanisms in text editors 5. Browser back button history

Example – Balanced Parentheses using Stack: Input: "{([])}" Algorithm: push on '(', '{', '['; pop and match on ')', '}', ']' If stack is empty at end and every match succeeded $\rightarrow$ balanced Time: $O(n)$, Space: $O(n)$

5. Queues

A queue is a linear data structure that follows FIFO (First In, First Out) principle. Think of a line at a bank: first person to arrive is first to be served.

Core Operations: - enqueue(x): add to rear — $O(1)$ - dequeue(): remove from front — $O(1)$ - front(): peek at front — $O(1)$

Variants: 1. Circular Queue: uses array with wrap-around to avoid wasted space 2. Deque (Double-Ended Queue): insert/delete from both ends 3. Priority Queue: element with highest priority dequeued first (implemented with heap)

Applications: 1. CPU scheduling (FIFO scheduling) 2. Breadth-First Search (BFS) traversal 3. Print spooling 4. Asynchronous data transfer (IO buffers)

6. Practice Problems

Problem 1: Implement a stack using two queues. What is the time complexity of each operation?

Problem 2: Given a string of brackets, determine if it is balanced. Example: "{[()]}" is balanced; "{[(())]}" is not.

Problem 3: Implement a queue using two stacks.

Problem 4: Reverse a linked list in-place. What is the space complexity?

Problem 5: Find the middle element of a linked list in one pass using two pointers (Floyd's tortoise and hare).

